White-Label Banking API: The Integration Handover

Zach Kelling

Lux Industries

2026

Abstract

This is the complete integration pack for a white-label banking tenant: one authenticated REST surface under `/v1/bank` covering accounts, balances, transfers, outbound payments, beneficiaries, cards, crypto, exchange, FX, and membership plans, plus the typed client (`@luxfi/bank`) and the interface standard (LP-3040). The surface is provider-neutral — the banking counterparty, card issuer, FX venue, and crypto backend are deployment choices that never appear in the client contract — and the sandbox serves the identical contract with seeded fixtures, deterministic settlement, and a testnet crypto faucet, so a tenant integrates end to end before any real-money credential exists.

1 The Platform

Environments. Two, same binary, same contract:

Environment	API base	Money
Sandbox	`https://api.sandbox.lux.financial/v1`	Seeded fixtures, testnet crypto, instant settlement
Production	`https://api.lux.financial/v1`	Live rails

Authentication. Every route requires `Authorization: Bearer <JWT>` issued by Lux IAM (lux.id) unless marked public. Ownership is enforced server-side: the authenticated principal must own every account-scoped record it touches. There is no API-key path on customer routes — IAM is the only authorizer.

White-label model. Branding (name, legal entity, theme, domain) is configuration, not code. A tenant ships its own brand on the same surface; the partner disclosure required on customer-facing pages is served by `GET /v1/bank/config`.

Conventions. Amounts are integers in minor units; every balance and transaction carries `decimals` (2 for most fiat, 0 for JPY, 6 for crypto). Errors return {`"error": "..."`} or {`"message": "..."`} with the matching HTTP status. Additive response fields must be tolerated.

2 Public Surface

Method	Path	Returns
GET	`/v1/bank/health`	{`status, sandbox`}
GET	`/v1/bank/config`	Currencies, network, partner disclosure + legal links
GET	`/v1/bank/plans`	The membership ladder (Section 9)

3 Accounts, Balances, Dashboard

Method	Path	Purpose
POST	`/v1/bank/onboard`	Create the caller's account from the KYC profile
GET	`/v1/bank/overview`	Dashboard aggregate: account, balances, wallet, cards, activity
GET	`/v1/bank/account/summary`	The caller's accounts
GET	`/v1/bank/transactions`	Transaction stream
GET	`/v1/bank/accounts/:id/balances`	Per-currency {`available, held, decimals, kind, valueUsd`}
GET	`/v1/bank/accounts/:id/wallets`	On-chain wallets
GET	`/v1/bank/accounts/:id/transactions`	Account history, newest first

4 Money Movement

Method	Path	Purpose
POST	`/v1/bank/transfers`	Book transfer between two own accounts
POST	`/v1/bank/payments/outbound`	Payment to a registered beneficiary; the rail (ACH, SWIFT, SEPA, wire, check, P2P) is selected from the beneficiary record
GET/POST/DELETE	`/v1/bank/beneficiaries[/:id]`	Counterparty registry

Transfer body: {`fromAccountId, toAccountId, amount, currency, reference`} → {`debitId, creditId, status`}. Payment body: {`accountId, beneficiaryId, amount, currency, reference`} → {`transactionId, status`}.

Status lifecycle, enforced server-side — all other transitions are rejected:

```
pending -> processing -> completed   ·   pending|processing -> failed | cancelled
```

Debits hold funds atomically at creation; completion releases the hold; failure or cancellation reverses it. A plan on the account (Section 9) sets the daily and monthly transaction limits.

5 Cards

The card ledger is immediate; issuance runs the card issuer's own account, KYC, and consent lifecycle behind a provider-neutral contract.

Method	Path	Purpose
GET/POST	`/v1/bank/cards`	List; issue on an account
POST	`/v1/bank/cards/:id/freeze` · `/unfreeze`	Card state
POST	`/v1/bank/cards/account`	Open the issuer card account (cardholder profile)
GET	`/v1/bank/cards/account`	Issuer account state: status, KYC, `virtualAccount`, cards
GET	`/v1/bank/cards/kyc`	Issuer KYC status
POST	`/v1/bank/cards/virtual`	Create the virtual-card account
GET	`/v1/bank/cards/virtual/kyc-url`	Hosted KYC session URL
GET	`/v1/bank/cards/virtual/consent-url`	Consent agreement URL
POST	`/v1/bank/cards/virtual/order`	Order the card once approved

Branch *only* on the normalized `status` + `nextAction` pair:

`status`	`nextAction`	Client action
`not_started`	`register`	`POST .../virtual`
`pending`	`complete_kyc`	Fetch KYC URL, open in the user's browser, poll
`pending`	`accept_agreement`	Fetch consent URL, open, poll
`pending`	`none`	Wait and poll
`rejected`	`retry_kyc`	Fresh KYC URL, retry, poll
`approved`	`order_card`	`POST .../virtual/order`
`error`	`retry_kyc`	Retry; support if persistent

The hosted URLs are sensitive: hand them straight to the user's browser; never log, persist, or send them to analytics. Platform KYC and issuer KYC never substitute for each other. In sandbox the issuer is simulated (`approved` / `order_card`), so the flow is exercisable with no upstream credential.

6 Crypto

Every account carries a non-custodial wallet (`lux-mainnet`; `lux-testnet` in sandbox).

Method	Path	Purpose
GET	`/v1/bank/wallet`	Wallet, address, network, crypto holdings
GET	`/v1/bank/crypto/prices`	Reference prices
POST	`/v1/bank/crypto/send`	On-chain send: {`asset, amount, toAddress`} → `txHash` + balances
POST	`/v1/bank/crypto/deposit`	Sandbox faucet ($25k equivalent cap per request)

The full loop — receive, hold, convert, send — runs end to end in sandbox. Conversion between fiat and crypto is the exchange below; the banking counterparty only ever sees ordinary fiat.

7 Exchange and FX

Method	Path	Purpose
POST	`/v1/bank/exchange/quote`	Quote any supported pair — fiat/fiat, fiat/crypto, crypto/crypto: {`fromCurrency, toCurrency, amount`}
POST	`/v1/bank/exchange/execute`	Execute; returns amounts, rate, and the updated balances
POST	`/v1/bank/fx/quote`	Dealt FX quote with TTL and `quoteId`
POST	`/v1/bank/fx/execute`	Lock in a quote: {`accountId, quoteId`}

8 Webhooks

Inbound provider events post to `/v1/bank/webhooks/*` and are authenticated by HMAC-SHA256: `X-Signature` = hex(HMAC-SHA256(secret, body)). Tenant-facing event delivery uses the same signing scheme.

9 Membership Plans

`GET /v1/bank/plans` publishes the ladder — one pricing source for every surface that renders plans:

Plan	Monthly	Card	IBAN	FX	Wires	Limits (day / month)
Silver	$29	Virtual	—	2.25%	$95 each	$10k / $100k
Gold	$99	Plastic	✓	1.95%	1 free	$50k / $500k
Black	$299	Metal	✓	1.45%	3 free	$250k / $2.5M
Sovereign	$999 · invite	Metal	✓	0.95%	10 free	$1M / $10M

A `plan` on the account overrides the entity-type transaction limits. Tenants may publish their own ladder; the mechanism is the same.

10 The SDK

`@luxfi/bank` (npm) is the typed client over the whole surface — zero dependencies, works in browser and server runtimes, errors throw `BankError` with status and parsed body:

```
import { Bank } from '@luxfi/bank'

const bank = new Bank({
  baseUrl: 'https://api.sandbox.lux.financial',
  token: () => store.token,
})

const plans = await bank.plans()
const { holdings } = await bank.wallet()
await bank.depositCrypto('BTC', 100_000)       // sandbox faucet, 0.1 BTC
await bank.sendCrypto('BTC', 40_000, 'bc1q...')  // returns txHash
```

```
const { data } = await bank.cardAccount()
if (data.virtualAccount?.nextAction === 'complete_kyc') {
  const { url } = await bank.cardKYCURL()      // straight to the browser
  window.open(url)
}
```

11 Integration Path

Sandbox first: seed credentials arrive at kick-off, the demo login mints a session, and every flow in this pack — onboarding, transfers, payments, the card lifecycle, the crypto loop, exchange — runs against `api.sandbox.lux.financial` with no upstream dependency. Production credentials issue after the end-to-end test journey passes; the cutover is a base-URL change. The interface standard is LP-3040; the living reference is `docs.lux.financial`.